# 004 — Docker Cheatsheet

> 🐋 Docker packages your application and all its dependencies into a **container** that runs consistently on any machine. This cheatsheet covers everything you'll use during the bootcamp.

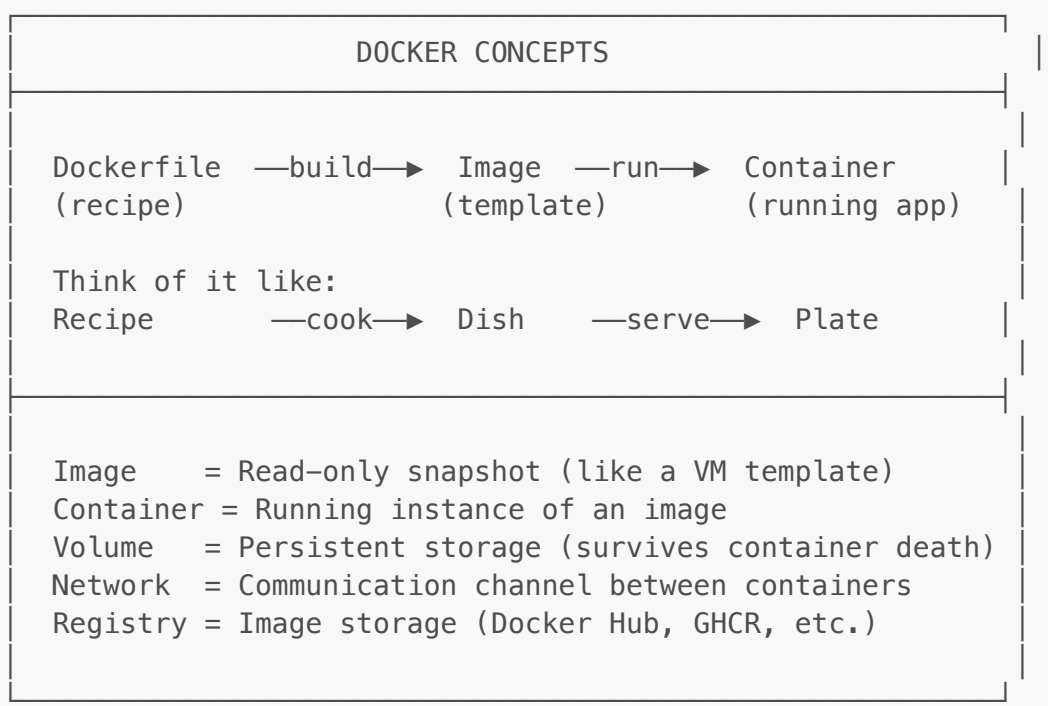
---

## Table of Contents

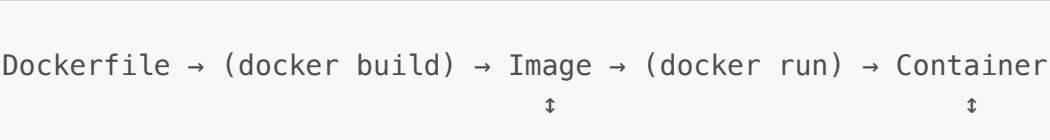


---

## 1. Core Concepts




### Lifecycle



```
                              (docker push/pull)      (start/stop/rm)
                                    ↕
                               Registry (GHCR)
```

---

## 2. Docker Images

### 2.1 Building Images

| Command | Description |
|---|---|
| `docker build -t myapp .` | Build image from Dockerfile in current dir |
| `docker build -t myapp:v1 .` | Build with specific tag |
| `docker build -f Dockerfile.prod .` | Build with specific Dockerfile |
| `docker build --no-cache -t myapp .` | Build without using cache |
| `docker build --platform linux/amd64 -t myapp .` | Build for specific platform |

**Example — Building the bootcamp API:**

```
docker build -t kanban-api -f services/api-dotnet/Dockerfile .
```

**Example — Build and push in one command (buildx):**

```
docker buildx build --platform linux/amd64 --push \
  -t ghcr.io/proxeon/bootcamp/api:latest \
  -f services/api-dotnet/Dockerfile .
```

### 2.2 Listing & Managing Images

| Command | Description |
|---|---|
| `docker images` | List all local images |
| `docker images -a` | List all images (including intermediate) |
| `docker image ls` | Same as `docker images` |
| `docker image rm myapp` | Remove an image |
| `docker rmi myapp` | Shorthand for remove |
| `docker rmi $(docker images -q)` | Remove ALL images ⚠️ |
| `docker image inspect myapp` | Show image details (layers, env, etc.) |

| Command | Description |
|---|---|
| `docker image history myapp` | Show image layer history |

### 2.3 Pulling & Pushing Images

| Command | Description |
|---|---|
| `docker pull nginx:latest` | Download image from registry |
| `docker pull ghcr.io/proxeon/bootcamp/api:latest` | Pull from GHCR |
| `docker push myapp:latest` | Push image to registry |
| `docker tag myapp:latest ghcr.io/user/myapp:latest` | Tag image for registry |

---

## 3. Docker Containers

### 3.1 Running Containers

| Command | Description |
|---|---|
| `docker run nginx` | Run container (foreground) |
| `docker run -d nginx` | Run in background (detached) |
| `docker run -d --name web nginx` | Run with custom name |
| `docker run -d -p 8080:80 nginx` | Map host port 8080 → container port 80 |
| `docker run -d -p 3000:3000 myapp` | Expose app on port 3000 |
| `docker run --rm nginx` | Auto-remove container when it exits |
| `docker run -it ubuntu bash` | Run interactive terminal |
| `docker run -d --restart always nginx` | Auto-restart on failure/reboot |
| `docker run -e "DB_HOST=localhost" myapp` | Pass environment variable |
| `docker run --env-file .env myapp` | Pass env file |
| `docker run -v ./data:/app/data myapp` | Mount host directory |
| `docker run --network mynet myapp` | Connect to specific network |

**Port Mapping Explained:**

```
-p HOST_PORT:CONTAINER_PORT

-p 8080:80     → Access container's port 80 via localhost:8080
-p 3000:3000   → Access container's port 3000 via localhost:3000
-p 443:443     → Access container's port 443 via localhost:443
```

### 3.2 Listing Containers

| Command | Description |
|---|---|
| `docker ps` | List running containers |
| `docker ps -a` | List ALL containers (including stopped) |
| `docker ps -q` | List only container IDs |
| `docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"` | Custom format |

**Example output:**

```
CONTAINER ID   IMAGE          COMMAND       STATUS         PORTS
NAMES
a1b2c3d4e5f6   nginx:latest   "nginx"       Up 2 hours     0.0.0.0:80-
>80/tcp     web
f6e5d4c3b2a1   postgres:17    "postgres"    Up 2 hours     0.0.0.0:5432-
>5432/tcp db
```

### 3.3 Stopping & Removing Containers

| Command | Description |
|---|---|
| `docker stop CONTAINER` | Gracefully stop (sends SIGTERM) |
| `docker stop $(docker ps -q)` | Stop ALL running containers |
| `docker kill CONTAINER` | Force stop (sends SIGKILL) |
| `docker rm CONTAINER` | Remove stopped container |
| `docker rm -f CONTAINER` | Force remove (even if running) |
| `docker rm $(docker ps -aq)` | Remove ALL containers ⚠️ |
| `docker start CONTAINER` | Start a stopped container |
| `docker restart CONTAINER` | Restart a container |

### 3.4 Container Logs

| Command | Description |
|---|---|
| `docker logs CONTAINER` | Show all logs |
| `docker logs -f CONTAINER` | Follow logs (live stream) |
| `docker logs --tail 50 CONTAINER` | Show last 50 lines |
| `docker logs --tail 50 -f CONTAINER` | Last 50 + follow |

| Command | Description |
|---|---|
| `docker logs --since 5m CONTAINER` | Logs from last 5 minutes |
| `docker logs --since 1h CONTAINER` | Logs from last 1 hour |
| `docker logs --timestamps CONTAINER` | Show timestamps |

### 3.5 Executing Commands Inside Containers

| Command | Description |
|---|---|
| `docker exec -it CONTAINER bash` | Open bash shell inside container |
| `docker exec -it CONTAINER sh` | Open sh shell (if no bash) |
| `docker exec CONTAINER ls /app` | Run single command |
| `docker exec CONTAINER cat /etc/hosts` | View file inside container |
| `docker exec -it CONTAINER env` | View environment variables |
| `docker exec -u root CONTAINER command` | Run as root user |

**Example — Debug the API container:**

```
# Open a shell in the running API
docker exec -it kanban-api sh

# Check if database is reachable from inside the container
docker exec -it kanban-api ping postgres

# View the app's environment variables
docker exec kanban-api env | grep DATABASE
```

### 3.6 Container Inspection

| Command | Description |
|---|---|
| `docker inspect CONTAINER` | Full JSON details |
| `docker inspect --format '{{.State.Status}}' CONTAINER` | Get specific field |
| `docker inspect --format '{{.NetworkSettings.IPAddress}}' CONTAINER` | Get IP address |
| `docker stats` | Live resource usage (CPU, RAM, NET) |
| `docker stats --no-stream` | One-time snapshot of stats |
| `docker top CONTAINER` | Show running processes |
| `docker diff CONTAINER` | Show filesystem changes |

## 4. Docker Compose

> Docker Compose manages **multi-container applications** defined in a `docker-compose.yml` file. It's how we run the entire Kanban app (API + Web + Admin + DB + Gateway) with one command.

### 4.1 Essential Commands

| Command | Description |
|---|---|
| `docker compose up` | Create & start all services (foreground) |
| `docker compose up -d` | Create & start in background (detached) |
| `docker compose down` | Stop & remove containers + networks |
| `docker compose down -v` | Stop & remove including volumes ⚠️ |
| `docker compose stop` | Stop without removing |
| `docker compose start` | Start previously stopped services |
| `docker compose restart` | Restart all services |
| `docker compose ps` | List running services |
| `docker compose logs` | View logs of all services |
| `docker compose logs -f` | Follow all logs |
| `docker compose logs -f api` | Follow logs of specific service |
| `docker compose logs --tail 20 api` | Last 20 lines of api service |

### 4.2 Build & Update

| Command | Description |
|---|---|
| `docker compose build` | Build all services |
| `docker compose build api` | Build specific service |
| `docker compose build --no-cache` | Build without cache |
| `docker compose up -d --build` | Rebuild & restart |
| `docker compose pull` | Pull latest images |
| `docker compose up -d --pull always` | Pull & restart |

### 4.3 Using Specific Compose Files

| Command | Description |
|---|---|
| `docker compose -f docker-compose.yml up -d` | Use specific file |

| Command | Description |
| --- | --- |
| `docker compose -f docker-compose.prod.yml up -d` | Use prod file |
| `docker compose -f docker-compose.yml -f docker-compose.override.yml up -d` | Merge files |

**Bootcamp examples:**

```
# Local development (includes PostgreSQL + Adminer)
docker compose up -d

# Production-like (includes Caddy gateway)
docker compose -f docker-compose.prod.yml up -d
```

### 4.4 Service Management

| Command | Description |
| --- | --- |
| `docker compose up -d api` | Start only the api service |
| `docker compose restart web` | Restart only the web service |
| `docker compose stop admin` | Stop only the admin service |
| `docker compose rm api` | Remove stopped api container |
| `docker compose exec api sh` | Shell into api container |
| `docker compose exec postgres psql -U kanban` | Open PostgreSQL CLI |
| `docker compose run --rm api dotnet test` | Run one-off command |

### 4.5 Scaling & Status

| Command | Description |
| --- | --- |
| `docker compose ps` | Show status of all services |
| `docker compose top` | Show processes in all services |
| `docker compose config` | Validate & show resolved config |
| `docker compose images` | List images used by services |
| `docker compose port web 3000` | Show public port mapping |

### 4.6 Docker Compose File Structure (Reference)

```
# docker-compose.yml
services:
  # Service name (used as hostname in Docker network)
```

```yaml
  api:
    # Build from Dockerfile
    build:
      context: .
      dockerfile: services/api-dotnet/Dockerfile
    # OR use pre-built image
    image: ghcr.io/proxeon/bootcamp/api:latest

    container_name: kanban-api        # Custom container name
    restart: always                    # Restart policy

    ports:
      - "5010:5010"                   # HOST:CONTAINER

    environment:                       # Environment variables
      -
DATABASE_URL=Host=postgres;Port=5432;Database=kanban;Username=kanban;Passwo
rd=secret
      - ASPNETCORE_ENVIRONMENT=Production

    env_file:                          # Load from file
      - .env

    volumes:
      - ./data:/app/data              # Bind mount
      - app_data:/app/storage         # Named volume

    depends_on:                        # Start order
      postgres:
        condition: service_healthy

    networks:
      - app_net                        # Connect to network

    healthcheck:                       # Health monitoring
      test: ["CMD", "curl", "-f", "http://localhost:5010/health"]
      interval: 30s
      timeout: 10s
      retries: 3

volumes:
  app_data:                            # Named volume definition

networks:
  app_net:
    driver: bridge                     # Network driver
```

---

## 5. Debugging Containers

### 5.1 Container Won't Start

```
# Step 1: Check exit code and status
docker compose ps -a

# Step 2: Read the logs
docker compose logs api

# Step 3: Check if it's a build issue
docker compose build api 2>&1 | tail -20

# Step 4: Try running interactively to see the error
docker compose run --rm api sh
```

### 5.2 Container Keeps Restarting

```
# Check restart count and last exit code
docker inspect kanban-api --format '{{.RestartCount}} - Exit:
{{.State.ExitCode}}'

# Check recent logs
docker logs --tail 50 kanban-api

# Common exit codes:
#   0   = Exited normally
#   1   = Application error
#   137 = Killed (OOM or docker stop)
#   139 = Segfault
#   143 = Graceful termination (SIGTERM)
```

### 5.3 Container Running But App Not Working

```
# Check if the app is listening inside the container
docker exec kanban-api sh -c "curl -s localhost:5010/health || echo 'NOT
RESPONDING'"

# Check environment variables are correct
docker exec kanban-api env | sort

# Check if the app can reach the database
docker exec kanban-api sh -c "nc -zv postgres 5432 2>&1"

# Check DNS resolution between containers
docker exec kanban-api sh -c "nslookup postgres"

# View running processes
docker top kanban-api
```

### 5.4 Port Issues

```
# Check what's using a port on host
# macOS:
lsof -i :3000

# Linux/WSL:
ss -tlnp | grep 3000

# Check container port mappings
docker port kanban-api

# Verify port is exposed in container
docker inspect kanban-api --format '{{range $p, $conf :=
.NetworkSettings.Ports}}{{$p}} -> {{$conf}}{{"\n"}}{{end}}'
```

### 5.5 Network Issues Between Containers

```
# List networks
docker network ls

# Inspect network (see connected containers)
docker network inspect kanban-prod_default

# Test connectivity between containers
docker exec kanban-api ping kanban-postgres

# Check DNS resolution
docker exec kanban-api nslookup postgres

# Note: Containers in the same docker-compose file share a network
# They can reach each other by SERVICE NAME (not container_name)
```

### 5.6 Volume / File Issues

```
# List volumes
docker volume ls

# Inspect volume (see mount path on host)
docker volume inspect kanban-prod_postgres_data

# Check mounted files inside container
docker exec kanban-gateway cat /etc/caddy/Caddyfile

# Copy file from container to host
docker cp kanban-api:/app/appsettings.json ./debug-settings.json
```

```
# Copy file from host to container
docker cp ./Caddyfile kanban-gateway:/etc/caddy/Caddyfile
```

### 5.7 Resource Issues

```
# Real-time resource monitoring
docker stats

# Output:
# CONTAINER    CPU %   MEM USAGE / LIMIT     MEM %   NET I/O
# kanban-api   2.5%    145MiB / 1GiB         14%     1.2kB / 500B
# kanban-web   0.3%    80MiB / 1GiB          8%      800B / 200B

# Check if container was killed by OOM
docker inspect kanban-api --format '{{.State.OOMKilled}}'

# View container resource limits
docker inspect kanban-api --format '{{.HostConfig.Memory}}'
```

### 5.8 Complete Debug Session Example

```
echo "=== SERVICE STATUS ==="
docker compose ps

echo ""
echo "=== RECENT API LOGS ==="
docker compose logs --tail 30 api

echo ""
echo "=== API ENVIRONMENT ==="
docker exec kanban-api env | grep -E "(DATABASE|ASPNET|CORS|COOKIE)" | sort

echo ""
echo "=== NETWORK CHECK ==="
docker exec kanban-api sh -c "nc -zv postgres 5432 2>&1 || echo 'DB
UNREACHABLE'"

echo ""
echo "=== HEALTH CHECK ==="
docker exec kanban-api sh -c "curl -sf http://localhost:5010/health && echo
' OK' || echo 'UNHEALTHY'"

echo ""
echo "=== RESOURCE USAGE ==="
docker stats --no-stream --format "table
{{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"
```

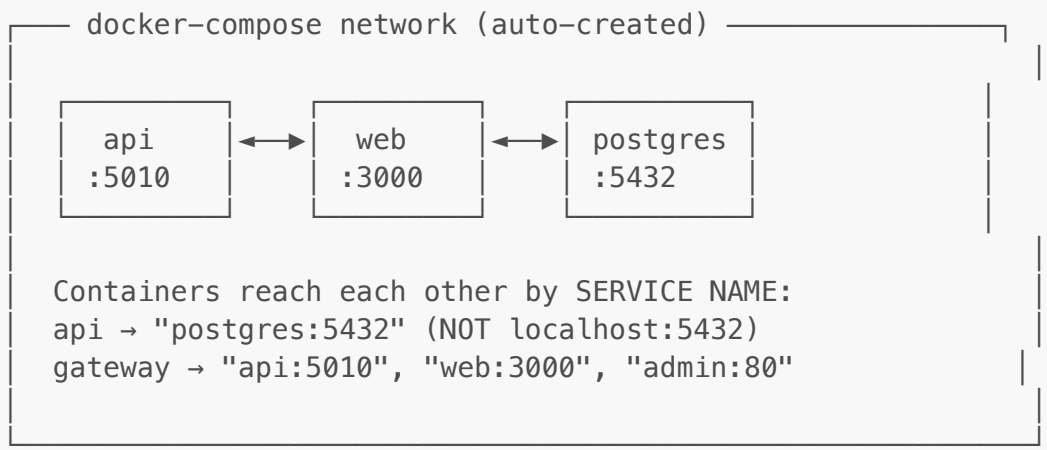
---

# 6. Docker Networks & Volumes

## 6.1 Networks

```
# List networks
docker network ls

# Create network
docker network create mynet

# Connect running container to network
docker network connect mynet kanban-api

# Disconnect container from network
docker network disconnect mynet kanban-api

# Remove network
docker network rm mynet

# Inspect (see which containers are connected)
docker network inspect mynet
```

**How containers communicate in Docker Compose:**




## 6.2 Volumes

```
# List volumes
docker volume ls

# Create volume
docker volume create mydata

# Inspect volume
docker volume inspect mydata
```

```
# Remove volume
docker volume rm mydata

# Remove unused volumes
docker volume prune

# Remove ALL volumes (DANGEROUS)
docker volume prune -a
```

**Volume types:**

```
services:
  api:
    volumes:
      # Named volume (managed by Docker, persists)
      - postgres_data:/var/lib/postgresql/data

      # Bind mount (maps host directory into container)
      - ./Caddyfile:/etc/caddy/Caddyfile

      # Anonymous volume (auto-named, less manageable)
      - /app/node_modules

volumes:
  postgres_data:    # Named volume declaration
```

---

## 7. Docker Registry (GHCR)

> GitHub Container Registry (GHCR) stores our Docker images. URL format:
> `ghcr.io/OWNER/REPO/IMAGE:TAG`

### 7.1 Login to GHCR

```
# Using Personal Access Token (PAT)
echo "ghp_your_token_here" | docker login ghcr.io -u YOUR_GITHUB_USERNAME --password-stdin
```

### 7.2 Push Images to GHCR

```
# Tag your local image for GHCR
docker tag kanban-api:latest ghcr.io/proxeon/bootcamp/api:latest

# Push
docker push ghcr.io/proxeon/bootcamp/api:latest
```

### 7.3 Pull Images from GHCR

```
# Pull specific image
docker pull ghcr.io/proxeon/bootcamp/api:latest

# Pull all images defined in docker-compose
docker compose pull
```

### 7.4 Build & Push (One Step with Buildx)

```
docker buildx build --platform linux/amd64 --push \
  -t ghcr.io/proxeon/bootcamp/api:latest \
  -f services/api-dotnet/Dockerfile .
```

---

## 8. Cleaning Up

> ⚠️ Docker can consume significant disk space. Clean up regularly.

### 8.1 Check Disk Usage

```
docker system df
```

**Example output:**

```
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          15        3         4.2GB     3.1GB (73%)
Containers      5         3         250MB     200MB (80%)
Local Volumes   8         2         1.5GB     900MB (60%)
Build Cache     --        --        2.1GB     2.1GB
```

### 8.2 Remove Unused Resources

| Command | What it removes |
|---|---|
| `docker system prune` | Stopped containers + unused networks + dangling images |
| `docker system prune -a` | Above + ALL unused images (not just dangling) |
| `docker system prune -a --volumes` | Above + unused volumes ⚠️ **DATA LOSS** |
| `docker image prune` | Dangling images only |

| Command | What it removes |
|---|---|
| `docker image prune -a` | All unused images |
| `docker container prune` | All stopped containers |
| `docker volume prune` | All unused volumes ⚠️ |
| `docker network prune` | All unused networks |
| `docker builder prune` | Build cache |
| `docker builder prune -a` | ALL build cache |

### 8.3 Selective Cleanup

```
# Remove specific containers
docker rm container1 container2

# Remove all containers matching a pattern
docker rm $(docker ps -a --filter "name=kanban" -q)

# Remove images by pattern
docker rmi $(docker images "ghcr.io/proxeon/*" -q)

# Remove old images (dangling = untagged intermediate layers)
docker image prune

# Remove stopped containers older than 24h
docker container prune --filter "until=24h"
```

### 8.4 Nuclear Option (Start Fresh)

```
# ⚠️ REMOVES EVERYTHING — containers, images, volumes, networks, cache
docker system prune -a --volumes -f
docker builder prune -a -f

# Verify clean slate
docker system df
```

### 8.5 Docker Compose Cleanup

```
# Stop and remove containers + default network
docker compose down

# Stop and remove containers + networks + volumes
docker compose down -v

# Stop and remove containers + networks + volumes + images
```

```
docker compose down -v --rmi all

# Remove specific service's container and image
docker compose rm -sf api
docker rmi $(docker compose images api -q)
```

---

## 9. Dockerfile Reference

### 9.1 Common Instructions

| Instruction | Purpose | Example |
|---|---|---|
| `FROM` | Base image | `FROM node:20-slim` |
| `WORKDIR` | Set working directory | `WORKDIR /app` |
| `COPY` | Copy files from host | `COPY package.json .` |
| `RUN` | Execute command during build | `RUN npm install` |
| `ENV` | Set environment variable | `ENV NODE_ENV=production` |
| `EXPOSE` | Document port (doesn't publish) | `EXPOSE 3000` |
| `CMD` | Default command when container starts | `CMD ["node", "server.js"]` |
| `ENTRYPOINT` | Fixed command (CMD becomes args) | `ENTRYPOINT ["dotnet", "app.dll"]` |
| `ARG` | Build-time variable | `ARG VERSION=latest` |
| `VOLUME` | Declare mount point | `VOLUME /data` |

### 9.2 Multi-Stage Build Pattern

Used in this bootcamp for smaller production images:

```
# Stage 1: BUILD (large image with dev tools)
FROM node:20-slim AS builder
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install
COPY . .
RUN pnpm build

# Stage 2: RUN (small image, only production files)
FROM node:20-slim AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

**Benefits:**

- Build tools don't end up in production image
- Final image is much smaller (100MB vs 1GB+)
- Fewer security vulnerabilities

### 9.3 Bootcamp Dockerfiles Explained

**.NET API** (`services/api-dotnet/Dockerfile`):

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS builder    # Big SDK for building
WORKDIR /src
COPY services/api-dotnet/ services/api-dotnet/
COPY contracts/ contracts/
WORKDIR /src/services/api-dotnet
RUN dotnet restore
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runner  # Small runtime only
WORKDIR /app
COPY --from=builder /app/publish .
EXPOSE 5010
ENTRYPOINT ["dotnet", "api-dotnet.dll"]
```

**Next.js Web** (`apps/web/Dockerfile`):

```
FROM node:20-slim AS builder
RUN corepack enable
WORKDIR /app
COPY . .
RUN pnpm install --force
```

```
RUN pnpm turbo run build --filter=web

FROM node:20-slim AS runner
ENV NODE_ENV=production
COPY --from=builder /app/apps/web/.next/standalone ./
COPY --from=builder /app/apps/web/.next/static ./apps/web/.next/static
EXPOSE 3000
CMD ["node", "apps/web/server.js"]
```

**Vite Admin** (`apps/admin/Dockerfile`):

```
FROM node:20-slim AS builder
RUN corepack enable
WORKDIR /app
COPY . .
RUN pnpm install --force
RUN pnpm turbo run build --filter=admin

FROM caddy:2-alpine AS runner          # Caddy serves static files
COPY --from=builder /app/apps/admin/dist ./admin
COPY apps/admin/Caddyfile /etc/caddy/Caddyfile
EXPOSE 80
```

---

## 10. Quick Reference Card

### Most Used Commands

```
# ─── BUILD ──────────────────────────────────────────
docker build -t myapp .
docker compose build

# ─── RUN ────────────────────────────────────────────
docker compose up -d              # Start all services
docker compose up -d --build      # Rebuild & start
docker run -d -p 3000:3000 myapp  # Run single container

# ─── STATUS ─────────────────────────────────────────
docker compose ps                 # Service status
docker ps                         # Running containers
docker stats                      # Live resource usage

# ─── LOGS ───────────────────────────────────────────
docker compose logs -f            # Follow all logs
docker compose logs -f api        # Follow one service
docker logs --tail 50 -f NAME     # Last 50 + follow

# ─── DEBUG ──────────────────────────────────────────
docker exec -it CONTAINER sh      # Shell into container
```

```
docker exec CONTAINER env          # View env vars
docker inspect CONTAINER           # Full details

# ── STOP ──────────────────────────────────────────
docker compose down                # Stop & remove
docker compose down -v             # Stop & remove + volumes
docker stop CONTAINER              # Stop one container

# ── CLEAN ─────────────────────────────────────────
docker system prune -a             # Remove all unused
docker builder prune -a            # Clear build cache

# ── REGISTRY ──────────────────────────────────────
echo $CR_PAT | docker login ghcr.io -u USER --password-stdin
docker compose pull                # Pull latest images
docker push ghcr.io/org/repo/img   # Push image
```

Bootcamp-Specific Commands

```
# ── LOCAL DEVELOPMENT ──────────────────────────────

# Start database only
docker compose up -d

# Start full production stack locally
docker compose -f docker-compose.prod.yml up -d

# View the app
# Web:   http://localhost/
# Admin: http://localhost/admin/
# API:   http://localhost/api/

# ── VPS DEPLOYMENT ─────────────────────────────────

# SSH into VPS
ssh vps

# Navigate to project
cd ~/kanban-prod

# Login to GHCR
echo $CR_PAT | docker login ghcr.io -u $GH_USERNAME --password-stdin

# Start services
docker compose up -d

# Check everything is running
docker compose ps

# View logs if something is wrong
docker compose logs -f
```

```
# Update after new images are pushed
docker compose pull && docker compose up -d

# ─── COMMON FIXES ──────────────────────────────────────

# Restart a misbehaving service
docker compose restart api

# Rebuild if code changed
docker compose up -d --build api

# Reset database (WARNING: deletes all data)
docker compose down -v
docker compose up -d

# Check SSL certificate generation
docker compose logs gateway | grep "certificate"
```

Environment Variable Patterns

```
# Pass single variable
docker run -e DATABASE_URL="..." myapp

# Pass from host environment
export DATABASE_URL="..."
docker run -e DATABASE_URL myapp

# Use .env file
docker run --env-file .env myapp

# In docker-compose.yml:
services:
  api:
    environment:
      - DATABASE_URL=${DATABASE_URL}     # From .env or host
      - ASPNETCORE_ENVIRONMENT=Production # Hardcoded value
    env_file:
      - .env                             # Load entire file
```

Useful Aliases (Add to ~/.bashrc or ~/.zshrc)

```
# Docker shortcuts
alias d='docker'
alias dc='docker compose'
alias dps='docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"'
alias dlogs='docker compose logs -f'
alias dexec='docker exec -it'
alias dclean='docker system prune -af && docker builder prune -af'
```

```
# Bootcamp-specific
alias deploy-up='cd ~/kanban-prod && docker compose pull && docker compose
up -d'
alias deploy-logs='cd ~/kanban-prod && docker compose logs -f'
alias deploy-status='cd ~/kanban-prod && docker compose ps'
```

---

## Diagram: Bootcamp Container Architecture